The Journal of Systems and Software 230 (2025) 112496

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

In Practice

Refactoring cross-project code duplication in an industrial software product line: A case study from RDW☆

Jesper van der Zwaag [a,b], Frank Driesens [b], Bouwe Postma [b], Andrea Capiluppi [a,*]

[a] Department of Computing Science, University of Groningen, The Netherlands
[b] RDW, Groningen, The Netherlands

ARTICLE INFO

Keywords:
Code duplication
Action research
Software product lines
Refactoring
Maintainability
Reusability

ABSTRACT

Code duplication across projects is a common challenge in Software Product Line (SPL) development, impacting maintainability and efficiency. This paper reports on an applied research project conducted within RDW, The Netherlands Vehicle Authority, aimed at addressing cross-project code duplication in a multi-application SPL environment.

The objective of this work was to investigate the impact of refactoring cross-project duplicated code on maintainability, code reuse, and technical debt within RDW's Electronic Services System (EDS), composed of 62 applications.

We used an action research approach, focusing on two applications to identify, refactor, and package duplicated features into reusable components. SonarQube was used to detect and quantify duplication, and the effectiveness of refactoring was evaluated using code duplication reduction metrics and developer feedback.

The refactoring effort led to the removal of 38% of the duplicated code on average, improving code reuse by 33%. Efficiency gains were observed in the second application due to the experience from the first intervention, reducing the time required for code maintenance and updates by 89%.

While the study demonstrates that systematically addressing cross-project code duplication in SPLs improves maintainability and reduces technical debt, it also highlights a key limitation: refactoring efforts need to be undertaken early to be feasible. As the number of applications in RDW's system steadily expanded to 80, the engineers reported that the required refactoring work, although highly beneficial, has become too extensive and time-consuming to be practical. The effort was ultimately left too late, making large-scale refactoring no longer a viable solution. Future strategies should focus on early duplication identification, architectural solutions such as modularization or microservices, and automated dependency management to prevent similar challenges from arising.

1. Introduction

Code duplication across projects is a prevalent issue in SPL development, often leading to challenges related to maintainability, efficiency and technical debt (Kästner et al., 2013; Tsantalis et al., 2015). In SPL environments, where multiple products share common functionality, code reuse is typically achieved through shared core assets. However, when duplication occurs, the same functionality is copied across different applications, leading to increased complexity in maintaining and evolving the software. Several studies indicate that approximately 5% to 20% of a software system's code may consist of duplicated fragments, often created by copying and pasting existing code, with or without minor modifications (BenIdris, 2020; Roy and Cordy, 2007). Cloned files are especially prone to accumulating technical debt and require more frequent updates, further straining maintainability (Lerina and Nardi, 2019). Research has demonstrated that addressing code duplication can reduce maintenance effort by up to 7% (Arif and Rana, 2020), which is critical for large-scale system management.

At the RDW (Netherlands Vehicle Authority), the Electronic Services System (EDS) consists of 62 applications developed as part of a multi-application SPL. These applications share core components but also exhibit significant code duplication across projects, resulting in technical debt and increasing the effort required for maintenance and updates. High rates of code cloning have been reported in industrial settings, particularly where ad-hoc reuse practices such as copy-pasting are common (Inoue et al., 2010; Gasparic et al., 2015). This challenge

☆ Editor: Daniel Mendez.

* Corresponding author.
E-mail address: a.capiluppi@rug.nl (A. Capiluppi).

https://doi.org/10.1016/j.jss.2025.112496
Received 26 November 2024; Received in revised form 9 May 2025; Accepted 10 May 2025
Available online 9 June 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

is evident in RDW's EDS, where cross-project code duplication is prevalent due to similar practices, as we detail in this study. Within RDW, new applications are frequently created by cloning and modifying existing ones, a practice often referred to as 'clone-and-own' (Dubinsky et al., 2013). Although cloning may provide short-term development efficiency, it leads to decreased long-term maintainability and increased technical debt if not properly managed (Monden et al., 2002; Zhang et al., 2012): while this approach has allowed rapid development in RDW, it has also led to significant cross-project code duplication and technical debt as the system has expanded. As a result, the EDS team launched a refactoring effort, realizing that maintaining 62 growing applications with years of unchecked code duplication had become too difficult, requiring urgent action to reduce technical debt and keep services running smoothly.

The rationale behind this research was to empirically evaluate whether systematic refactoring of duplicated code in a large-scale SPL environment not only reduces redundancy, but also leads to measurable improvements in software maintainability, despite the lack of direct assessments of maintenance effort in prior studies. Moreover, we wanted to investigate how the experience gained in refactoring one application could potentially influence the efficiency of subsequent interventions: large-scale studies have shown that while individual refactorings may not always have a dramatic impact, sustained efforts can significantly improve maintainability and reduce technical debt (Szőke et al., 2017). Our study aims to provide key insights and practical recommendations for software practitioners in SPL environments, particularly regarding the importance of early identification of duplication, the use of automated tools and the creation of reusable components across project boundaries.

This paper reports on an applied research project conducted at RDW, aimed at reducing code duplication across projects within the EDS by refactoring duplicated features into reusable components. An action research approach was employed to guide the intervention, focusing on two selected applications. Action research has been widely used in industry settings for fostering collaboration and improving software processes (Petersen et al., 2014; Staron, 2019). The research sought to quantify the impact of refactoring on maintainability, code reuse and technical debt reduction, with tools such as SonarQube used to detect and measure duplication. SonarQube, commonly used in industry for detecting technical debt, is well-suited for identifying duplicated code across projects (Rantala et al., 2024).

This research was guided by the following research questions:

RQ1. How does refactoring cross-project code duplication in a multi-application SPL environment impact code maintainability and technical debt?

RQ2. What are the measurable effects of reducing code duplication on maintainability, efficiency and code reuse within the RDW's EDS?

RQ3. How does experience gained from refactoring one application influence the efficiency and effectiveness of subsequent interventions?

This paper is structured as follows: Section 2 presents a review of the related literature on service health metrics, SOA migration, and empirical software engineering. The research methodology, including the action research protocol and data collection approach, is detailed in Section 3. The phases of the action research are addressed in distinct sections: 'Identifying the problem' (Section 4), 'Planning' (Section 5), 'Action' (Section 6), 'Observation' (Section 7) and 'Reflection' (Section 8). Section 9 provides an in-depth interpretation of the findings, and the implications of the findings for practitioners outside of RDW's studied domain. Section 10 discusses the main threats to validity, while Section 11 concludes the paper by summarizing key findings, discussing limitations, and suggesting future research directions.

2. Related work

2.1. Code duplication

Code duplication is a pervasive problem, affecting between 5% and 20% of software systems (Roy and Cordy, 2007). It significantly contributes to technical debt by embedding poor coding practices that increase future maintenance costs (BenIdris, 2020). Studies have shown that cloned files are not only more prone to changes but also tend to accumulate higher levels of technical debt (Lerina and Nardi, 2019). Moreover, self-admitted technical debt, commonly found in code comments, often persists for extended periods, highlighting its long-lived nature (Bavota and Russo, 2016). This issue arises from several recurring causes, including tight deadlines, inadequate planning and insufficient technical knowledge among developers (BenIdris, 2020).

Code duplication strongly correlates with technical debt and reduced maintainability. Files with clones have higher technical debt and are more prone to changes (Lerina and Nardi, 2019). In model-driven engineering, duplicated code can accumulate more technical debt than handwritten code (He et al., 2016). Addressing duplication can reduce maintenance effort by up to 7% (Arif and Rana, 2020). Furthermore, this is especially crucial in machine learning systems, where duplication impacts configurations and models (Tang et al., 2021).

High rates of code cloning have been reported in industrial settings (Inoue et al., 2010), with ad-hoc reuse via copy-pasting (Kapser and Godfrey, 2003), project reuse practices (Gasparic et al., 2015) and agile testing methods (Hasanain et al., 2018) cited as common causes. Cloning occurs due to technical, personal and organizational factors (Zhang et al., 2012). Although it may enhance development efficiency (Gasparic et al., 2015), code cloning generally reduces maintainability (Monden et al., 2002), unless aligned with design patterns (Yoshida et al., 2012). Clone management must consider both technical and organizational contexts (Zhang et al., 2012), supported by effective detection techniques (Bellon et al., 2007).

Beyond increasing maintenance costs, code cloning is also considered defect-prone, as inconsistent modifications to duplicated code can lead to unintended behavior. However, the specific impact of inconsistent changes on program correctness has received limited investigation. A large-scale study found that such inconsistent modifications frequently introduce faults, underscoring the risks of unmanaged duplication (Juergens et al., 2009).

In this study, we identified and categorized duplicated code using the established taxonomy of clones (Type I–IV) (Rattan et al., 2013). While the taxonomy of clone types provides a structured approach to understand code duplication, their practical implications vary depending on the context of a SPL environment. In our study at RDW, we observed that Type I (exact copies) and Type II (renamed clones) dominated the duplicated code across the 62 applications, reflecting common reuse practices like copy-pasting with minimal renaming. Type III clones (near-miss duplicates with modifications) were less frequent but posed unique refactoring challenges due to application-specific variations, while Type IV clones (functional duplicates) were rare and typically beyond the scope of straightforward refactoring.

Our paper builds upon the studies that highlight code duplication as a driver of technical debt and decreased maintainability by focusing specifically on a cross-project, multi-application environment. The approach taken addressed the complexities of SPLs by not only identifying duplicated code but actively refactoring it into reusable components, thus reducing technical debt beyond what prior studies have accomplished.

2.2. Techniques to detect and deal with duplicated code

Cross-project code duplication is a serious maintenance concern and various methods have been developed to identify and manage code duplication. In general, recent research has long advocated holistic

technical debt management strategies (Alves et al., 2016), that is, a *detection* phase coupled to a *management* phase.

Clone detection relies on a range of techniques, each offering unique advantages in identifying and managing duplicated code (Ain et al., 2019). Program dependence graphs and slicing methods (Komondoor and Horwitz, 2001) analyze the structural relationships between code segments, while abstract syntax trees (Baxter et al., 1998) provide a hierarchical representation that facilitates the detection of syntactic similarities. Hybrid approaches (Ain et al., 2019) combine multiple techniques to enhance accuracy and robustness in identifying code clones.

Several tools have been developed to support clone detection and analysis: DUPLOC offers both visual and automated detection capabilities, helping developers quickly identify duplicated code across projects (Rieger and Ducasse, 1998). Siamese is a scalable, incremental code clone search tool that uses multiple code representations to improve accuracy and performance (Ragkhitwetsagul and Krinke, 2019). Other tools focus on improving clone comprehension by providing detailed insights into the nature and extent of duplication (Kapser and Godfrey, 2005). Beyond detection, visualization techniques such as tag clouds aid in understanding clone distribution, while refactoring strategies help manage and eliminate unnecessary duplication, ultimately improving software maintainability (Ragkhitwetsagul et al., 2018; Sano et al., 2014).

Tools like SonarQube are widely used to detect technical debt (Rantala et al., 2024), while reusing code from platforms such as StackOverflow can exacerbate this issue (Digkas et al., 2019). Static analysis tools are vital for improving software quality by identifying defects and duplications without execution (Stefanović et al., 2020; Nikolić et al., 2021). They complement traditional testing and focus on early bug detection, adherence to coding standards and performance optimization (Zheng et al., 2006; Pargaonkar, 2023), but false positives are a common limitation, particularly in educational settings (AlOmar et al., 2023a). Implementing static analysis tools requires strategic planning for optimal effectiveness (Stefanović et al., 2021), making them a valuable, affordable addition to fault detection (Gosain and Sharma, 2015).

Clone detection tools face challenges in large-scale projects, generating large result sets that can be difficult to manage (Kapser and Godfrey, 2005). These tools struggle to detect complex duplications (beyond simple copy–paste scenarios Bellon et al., 2007) and scalability remains a key issue in managing extensive software suites (Li et al., 2006). Despite these limitations, clone detection is crucial for maintenance, especially in the era of Big Data (Roy, 2018).

Dealing with code duplication has in general provided less conducive experiences: approaches like Formal Concept Analysis detect duplication and suggest refactoring (Torres, 2004), while dependency analysis combined with clone detection helps extract common libraries (Bauer and Hauptmann, 2013). Visualization techniques improve understanding of system-wide duplication (Rieger et al.) and machine learning algorithms like Convolutional Neural Networks (CNN) have offered promising solutions for recommending refactorings (AlOmar et al., 2023b). Tools like JDeodorant facilitate clone refactoring, particularly in Java projects (Mazinanian et al., 2016), though limitations exist in detecting complex duplication beyond copy–paste scenarios (Bellon et al., 2007). A multi-faceted approach remains the best strategy for effective code duplication refactoring.

While previous research primarily emphasizes detection techniques, such as program dependence graphs and hybrid approaches, our paper advances these methods by combining them with a refactoring framework tailored for large-scale, multi-application environments. This approach closes the gap in existing literature, where detection often lacks corresponding actionable strategies for managing and mitigating the duplication effectively.

2.3. Impact of refactoring on code duplication

The impact of refactoring on maintainability has not provided a firm statement of its effectiveness on maintainability. While some studies report positive effects (Szőke et al., 2017; Al Dallal and Abdin, 2017; Malhotra and Chug, 2016), others have shown mixed or inconclusive results (Szoke et al., 2015; Almogahed et al., 2021; Alshayeb, 2009). Large-scale industrial studies suggest that while individual refactorings may have minimal effect, sustained efforts can improve maintainability (Szőke et al., 2017). Automated tools have shown promise in improving code quality (Szoke et al., 2015), with evidence suggesting that refactoring enhances both quality and productivity in agile teams (Moser et al., 2008). However, the relationship between specific refactoring techniques and quality attributes remains unclear (Almogahed et al., 2021) and improper implementation can introduce new errors (Malhotra and Chug, 2016).

Also the research on refactoring's impact on software quality is mixed: some studies report improvements in complexity, maintainability and code size (Shrivastava and Shrivastava, 2008; Shatnawi and Li, 2011; Reddy and Rao, 2009). However, others find inconsistent results across internal and external quality measures (Kannangara and Wijayanake, 2015; Stroggylos and Spinellis, 2007). The effects vary depending on the refactoring operations and quality metrics examined (Chaparro et al., 2014; AlOmar et al., 2019). In mobile app development, refactoring generally enhances most metrics but may negatively affect others (Hamdi et al., 2021). Overall, while refactoring is believed to improve software quality, its effects are context-dependent and complex.

Prior studies offer mixed results on the maintainability benefits of refactoring; however, this paper demonstrates that systematic refactoring in an SPL context, through the packaging of reusable components, yields measurable improvements in maintainability and efficiency. Our study complements existing literature by quantifying these benefits with real-world developer feedback and efficiency gains, adding empirical support for refactoring as a viable strategy in complex, large-scale systems.

2.4. Action research in industry settings

Action research (AR) has become an important methodology for software engineering, fostering collaboration between academia and industry (Petersen et al., 2014; Staron, 2019). It is particularly useful for studying global software teams, offering flexibility and in-depth analysis of complex projects (Clear, 2009; Pino et al., 2013). AR is still underutilized in software engineering but is growing in popularity (Santos and Travassos, 2009). It combines qualitative and quantitative approaches (Erro-Garcés and Alfaro-Tanco, 2016), allowing researchers to engage with the research process and gain detailed insights (Farooq and O'Brien, 2014). However, AR studies need better reporting on iterations, collaboration models and rationale for decisions (Petersen et al., 2014).

AR has been proven effective for Software Process Improvements (SPI) across various organizational contexts, leading to significant quality enhancements (van der Schuur et al., 2011; Zarour et al., 2012). Key lessons include engaging SPI teams in reflection and theory (Borjesson, 2006), adapting interventions through multiple iterations (Petersen et al., 2014) and managing risks in SPI initiatives (Iversen et al., 2004). Collaborative practices are critical, particularly for small organizations (Sampaio et al., 2015; Santos and Travassos, 2009). Long-term success, as seen with NASA's Software Engineering Laboratory, requires sustained adaptation to evolving environments (Basili et al., 2002).

This paper uses action research to bridge the gap between theoretical refactoring benefits and practical implementation in a complex organizational setting. By iterating through cycles of planning, action, and reflection, the study aligns with action research methodologies but uniquely applies it within an SPL environment, providing a concrete model for refactoring that can be adapted by similar organizations. This approach addresses the need for iterative, reflective research in software engineering, as noted in previous studies.

3. Methodology

This section outlines the methodology employed in this study, describing the action research approach, the selection and description of the research site, the design of the intervention and the criteria used to evaluate its success. We also present the phases of the research cycle and the data collection and analysis methods.

Action research approach. This study employed an action research methodology, a participatory and iterative research approach that is well-suited for addressing practical issues in real-world settings (Petersen et al., 2014). Action research integrates reflection, collaboration and intervention, making it ideal for investigating and improving organizational processes through continuous feedback and empirical observation. The cyclical nature of this methodology involves stages of planning, action, observation and reflection, allowing for refinements based on the outcomes of each intervention cycle.

Site selection and description. The research was conducted at the Netherlands Vehicle Authority (RDW), focusing on its Electronic Services System (EDS). EDS consists of 62 applications developed within a Software Product Line (SPL) environment, where applications share core components but also exhibit significant cross-project code duplication. The frontend of each application is built using Angular, while the backend is implemented in C# .NET, with middleware components ensuring integration with on-premise systems and other cloud-hosted services.

This study examines RDW's EDS as a real-world case of industrial code duplication, using an action research approach to evaluate the effectiveness of systematic refactoring. By focusing on two representative applications, the research assesses the feasibility of reducing duplication through reusable components. Furthermore, RDW's transition to a microservices architecture provides key insights into the scalability challenges of large-scale refactoring, making this case study highly relevant to both academia and industry.

Each application follows a consistent architectural pattern, including dependency injection and modular structures, and provides one or more electronic services. These shared characteristics, combined with the frequent addition of new applications, have contributed to substantial duplication across projects, as identified using tools such as SonarQube. While specific quantitative details cannot be disclosed due to confidentiality restrictions, our findings underscore the systemic challenges of duplication in large-scale SPL environments.

Intervention design. The intervention focused on refactoring duplicated code from two selected applications within the EDS into reusable components. This refactoring was performed by extracting duplicated functionality and packaging it into shared libraries using NPM (for Angular code) and NuGet (for C# .NET code). The aim was to reduce cross-project code duplication, enhance maintainability and improve overall efficiency. The intervention followed the typical action research cycle of planning, action, observation and reflection.

Evaluation criteria. The intervention's success was evaluated through both quantitative and qualitative metrics. The key quantitative metric was the reduction in code duplication, as measured by SonarQube, which was used to scan the codebase before and after refactoring. The reduction in duplicated lines of code across projects served as a primary indicator of success. Qualitative measures included developer feedback and the assessment of the refactoring's impact on maintainability and ease of use.

Research cycle phases. The research was conducted in two cycles, each focusing on refactoring a different application within the EDS. Each of the action research cycles involved five phases:

- **Phase I: Identifying the problem**. During this phase, we performed an analysis of the applications to detect cross-project code duplication using SonarQube (see Section 4).
- **Phase II: Planning**. In this phase, we developed a refactoring plan to package duplicated code into reusable components, ensuring they can be integrated across different applications (Section 5).
- **Phase III: Action**. In this phase, we implemented the refactoring activities by extracting code and integrating shared libraries (Section 6).
- **Phase IV: Observation**. Measuring the reduction in duplicated code and gathering developer feedback (Section 7).
- **Phase V: Reflection**. Reflecting on the outcomes, lessons learned and improvements for subsequent cycles (Section 8).

Data collection and analysis. This study examines the impact of refactoring cross-project code duplication across three core software quality dimensions: technical debt (i.e., the accumulated design compromises that increase long-term maintenance costs), code reuse (i.e., utilizing existing, well-structured components to reduce redundancy and improve development efficiency), and maintainability (i.e., how easily can a system be modified to correct defects, improve performance, or adapt to new requirements). These three aspects are interdependent: reducing technical debt through systematic refactoring improves maintainability, while increased reuse streamlines future development.

Data were collected through SonarQube scans to measure code duplication before and after the intervention. Additionally, qualitative data in the form of developer feedback was gathered to assess the impact of refactoring on the maintainability and usability of the codebase. The combination of quantitative and qualitative analysis provided a comprehensive understanding of the intervention's effectiveness.

4. Phase I: Identifying the problem

In this phase, our focus was to analyze the extent, nature and causes of duplicated code across the EDS system to establish a clear understanding of the problem.

When the refactoring activity was planned, EDS controlled 62 different applications[1] that together provided 112 online services. All applications used the same core technologies: the browser application was made in Angular, which communicated almost exclusively with an application-specific API written in C# .NET, both hosted together as an Azure Web Service. The API handled all requests and communicated with other systems, such as other cloud-hosted services, authentication services, and the (on-premise) backend and database through a middleware, in this case, Microsoft BizTalk. This applied to all applications.

EDS used dependency injection to supply dependencies to all objects: RDW originally used `StructureMap`[2] to do this task, and about half of the applications still use it, albeit it is no longer maintained. The rest of the applications use Microsoft's AspNet dependency resolver, and each application contains one instance of the duplicated AspNet class. Parts of registering the dependencies in the `DefaultRegistery` are also duplicated.

Every application had a repository that contains the folders (see the template of the directory structure below):

```
Repository
├─ BizTalk/ -- Middleware code
├─ FrontEnd/ -- .NET API
├─ FrontEnd.App/ -- Angular frontend
├─ Pipelines/ -- CI/CD config
└─ Test/ -- Integration and end-to-end test code
```

[1] At the time of writing the number of applications has grown to 80.

[2] https://docs.particular.net/nservicebus/dependency-injection/structuremap

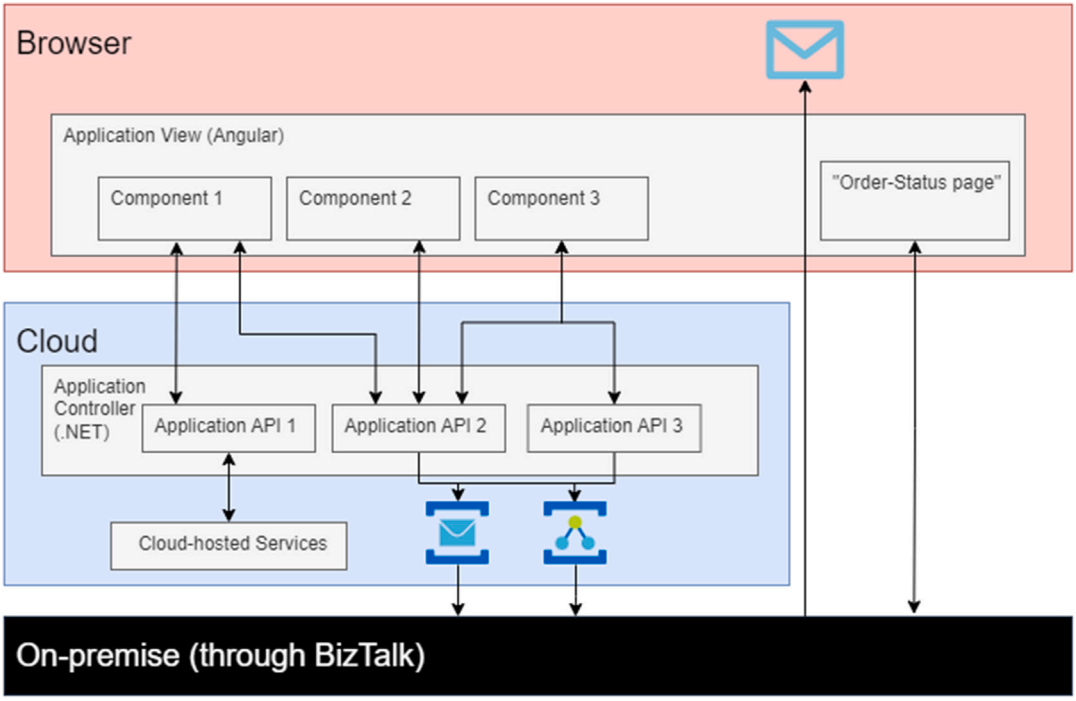

Fig. 1. An example architecture of an EDS Application.

When this refactoring work was planned, no automatic processes were set up to update all the applications when packages are updated. There was also no registration or list of what application uses which packages: the reason was that when a package gets updated, it was often a non-critical update. Any application that used the package, was then updating the package when the application got changed for some other reason. If the update was critical, to update all applications to the latest version of a package, then 'some script' was generated to aid with the process. The problem of having to manually update packages in multiple applications was tackled on a case-by-case basis and was done manually, or by creating a custom script.

To make working on so many different applications as easy as possible, EDS not only used the same technologies, but the applications also followed a similar flow and architecture. A simplified example architecture of an average EDS application is visualized in Fig. 1.

`BizTalk` (containing the files, code and schemas for the middleware), `FrontEnd`, `Frontend.App`, `Pipelines` (containing the settings and config for the CI/CD pipeline) and `Test` (with the integration and end-to-end tests).

During our intervention, we focused the refactoring on the Angular code in `FrontEnd.App` and the C# .NET code in the folder `FrontEnd` of each application. Together, they form an application that provides one or more services.

4.1. Angular

In this subsection, we will discuss the default Angular setup within the code base. Some elements are already (partially) packaged and some are not.

The general structure of the Angular code in `FrontEnd.App` is as follows:

```
FrontEnd.App/
├─ src/
│  ├─ app/
│  │  ├─ dienst/
│  │  │  ├─ *'dienst'-abbreviation*
│  │  │  │  └─ Main 'dienst' component, constants and
│  │  │  │     module
│  │  │  └─ shared/
│  │  │     └─ Components, models, services, etc. for
│  │  │        the 'dienst'
│  │  ├─ order-status-pages/ optional
│  │  │  └─ Separate folder for the status of the or-
│  │  │     der
│  │  └─ shared/
│  │     └─ Components, models, services, etc. not
│  │        'dienst'-specific
│  ├─ assets/
│  │  └─ 'Dienst'-specific images and files
│  └─ environments/
│     └─ Settings for the different OTAP environ-
│        ments
└─ Config and settings files
```

Services. In Angular, multiple services are used: a 'service' is a class that an application may need,[3] and it usually has a very small and distinct purpose. For example, RDW has the 'Postcode' (ZIP code) service in Angular, that provides the application with data linked to a postcode (e.g., city, street name). Services like this are not used as commonly as other components but are usually very similar, as their implementation does not differ between applications. Angular services are designed to be plugged into applications through dependency injection. These

[3] https://angular.io/guide/architecture-services

services should therefore be relatively straightforward to extract into a package.

Header and footer. The header and footer are elements that are used in every application. The EDS team already has a `layout` package, that contains a default header part (only the part with the logo), as well as a default footer. The footer is mostly used as is, while the header is too incomplete and is often surrounded with extra HTML and typescript, as it is currently just a logo with a bit of text, no `navbar` for example.

OrderStatus. The 'OrderStatus' is one of the most used components in the front-end. It is already largely packaged as an Angular package. Not all applications have an order status (page) as not all work with orders. Simple and direct request–response services, e.g., requesting information about a vehicle with its license plate number, do not work with orders. The package provides individual elements that can be used to create the order status page. This way, all styling and element definitions are defined in one place and all applications can use these elements to create their own order status page.

Contact and request information page. Most applications start with a contact information page and a page for entering information for the request. The contact information page often contains only an email field and a confirm-email field, but other information, e.g., name, telephone number, or location number might also be needed. The request information page is often application-specific. It can be a license plate, a specific code, vehicle information, or personal or company information.

Akkoord page. The page 'Akkoord' (Dutch for 'Agree') is a page where the user must agree with the information that the application will send. The page displays all the data that was either retrieved or entered, together with a checkbox that the user must check before the application will submit the request (or 'order' as it is often called). This allows the user to double-check the request or order before it is sent and potentially paid.

Bevestiging page. The page 'Bevestiging' (Dutch for 'Confirmation') is a page to confirm that the request or order has been sent and will be processed. It will also tell the user additional confirmation steps that will be taken, e.g., a confirmation by e-mail. The page usually does not contain information about the request or the submitted order. For that, the order status page is used. The confirmation email usually contains the order ID, as well as a direct link to the order status page.

4.2. C# .NET

This subsection discusses the (most significant) shared C# .NET elements within the EDS code base. Just like Angular, some elements are already packaged.

Controllers. In the applications, the `controllers` are used to define the API endpoints. It usually consists of a single function to accept the request, perform validation, logging and call the respective `process`. Most controllers are small and share a minimal base. Some controllers interact with packaged process classes. These controllers often have one or a few lines of code different in each application, so they were not packaged. By extracting these lines of code that can differ, we can also package the controllers.

Processes. The `processes` are called by the `controllers` and handle the interaction between the application and other systems. Some are packaged already. Many of these processes share the same design and perform similar actions, with different models. Other than packaging processes that are not application-specific, we might be able to also extract a base from the processes.

Policies. We also noticed that a `RetryPolicy` is often used by the processes when they communicate with the backend or other systems. This policy defines what to do in case of transient (i.e., self-resolving) errors, how many times to retry and how long to wait. There is essentially one configuration that is being used but the provider is copied a lot. We are likely able to make this its own shared feature.

The general structure of the C# .NET code in `FrontEnd` is as follows:

```
FrontEnd/
├─ Rdw.Eds.*dienst*.Models/
│  └─ Models of all objects in the service
├─ Rdw.Eds.*dienst*.Process/
│  ├─ Client/
│  ├─ Interfaces/
│  ├─ Policies/
│  └─ Submit and retrieve processes
├─ Rdw.Eds.*dienst*.Process.UnitTests/
├─ Rdw.Eds.*dienst*.Web/
│  ├─ Controllers/
│  ├─ DependencyResolution/
│  ├─ Handlers/
│  ├─ ViewModels/
│  └─ Startup and Utils files
└─ Rdw.Eds.*dienst*.Web.UnitTests/
```

4.3. SonarQube

EDS's CI/CD pipeline includes SonarQube for code analysis, primarily checking for duplication within individual applications. However, it does not detect cross-project duplication by default. While SonarQube offers this capability, enabling it to check for code duplication across the entire RDW ICT codebase (rather than isolating the EDS projects) would significantly increase analysis time due to the extensive size of the full codebase. Quantifying this increase was impractical within the study's scope, as it would require substantial resources to test across such a large system.

Despite evaluating other static analysis tools like Understand by Scitools[4] and Arcan,[5] SonarQube was preferred due to its seamless integration with RDW's existing pipeline, familiarity among developers, and superior ease of use. It allowed parameter tuning (e.g., minimum duplicated lines/characters) and cross-project duplication detection, although the latter was not initially enabled.

A limitation of SonarQube was its occasional failure to detect duplication when slight modifications were present between applications. To address this, we fine-tuned detection parameters by testing configurations across five representative projects, comparing results against known duplications within RDW/EDS and evaluating the relevance of other detected cases. The "minimum duplicated lines" was set as low as possible to maximize sensitivity, then "minimum tokens" was adjusted to filter out false positives (such as unrelated code flagged due to shared variable names or Angular hooks) ensuring mostly meaningful duplications were captured. Exact parameter values were not recorded, but the process prioritized comprehensive detection with minimal noise. Overlooked duplications, particularly in components duplicated across many projects, were manually identified by reviewing absences in other projects, often due to minor differences (e.g., a single altered line or varying code styles). These cases were infrequent, as SonarQube generally performed well in identifying duplicated code.

[4] https://scitools.com/
[5] https://www.arcan.tech/

Table 1
The 13 most duplicated Angular files/components (anonymized). The 'Projects' column indicates the number of projects with files containing duplicated lines, as detected by SonarQube.

File ID	Projects	Notes
File A	60	Contains the service name and basic loading text. It also includes references to browsers for fallback use if the application fails to load. The loading and reference content is repeated across multiple applications.
File B	51	Defines logic for the header, including text configuration based on user authentication, login information and a session countdown timer. These elements are duplicated across several projects.
File C	45	Specifies the main application routes, including paths for features like order status, a browser-closure page, empty-service handling and catch-all 404 errors. These routes often have similar structures.
File D	38	Represents a default step (`agree`) included in most services. Some services add extra code for specific fields and checks. Some implementations extend a base version of this file (referred to internally as File H-ext), which provides standard functions and dependency injection, but not all do.
File E	38	Serves as the main component for each service. Functions such as displaying a loading spinner, handling routing events and configuring return URLs are frequently duplicated across projects.
File F	29	Implements the order status page. While some variations exist among services, the foundational structure remains consistent across projects.
File G	25	Similar to File B, this file is not detected in all projects due to differences in styling (both visual and code-wise) across services.
File H	15	Represents another default step (`confirmed`) that is highly consistent across projects. However, minor modifications are present in some services. Some implementations extend a base version of this file (referred to internally as File H-ext), which provides standard functions and dependency injection, but not all do.
File I	13	Implements a default step (`request data`), with characteristics and duplication patterns similar to other default steps.
File J	13	Contains the main application component. Certain functions are repeated across multiple projects.
File K	12	Represents a default step (`agree`) with minor variations across services to account for service-specific user data. Despite these differences, a portion of the page is standardized across applications.
File L	11	Implements a default step (`contact data`), sharing similar characteristics and duplication patterns with other default steps.
File M	9	Represents a default step (`confirmed`) used for confirming orders. The file is highly consistent across services as it lacks service-specific data. However, minor differences like service names prevent broader detection.

FrontEnd.app. To get an idea of where the cross-project duplicated code problems lie in the FrontEnd.app, we ran a static code analysis with SonarQube on all 62 applications. We extracted the files that, according to SonarQube, have cross-project duplicated code in multiple applications. We sorted those by the number of projects the duplicated code was found in and did a manual preliminary analysis of the findings. These results can be found in Table 1.

FrontEnd. Similar to Angular, to get an idea of where the cross-project duplicated code problems lie in the C# .NET code, we ran SonarQube on all 62 applications. We extracted the files that, according to SonarQube, have cross-project duplicated code in multiple applications. We sorted those by the number of projects the duplicated code was found in and did a manual analysis of the findings. These results can be found in Table 2.

5. Phase II: Planning

This phase involved the selection of target applications for refactoring, based on the analysis of the root cause of the duplicated code, and guided by an impact and risk analysis to maximize benefits and minimize complexity.

5.1. Addressing the root causes of duplicated code

To understand how existing packages are utilized, we interviewed developers about their usage patterns. One developer highlighted the process of revamping an older application, designed for anonymous or business users to request information by entering a license plate. In this instance, not all existing packages were used due to limitations such as outdated functionality or mismatched requirements. Instead of updating or creating packages to meet specific needs, developers frequently chose to manually replicate functionality by writing custom code directly in the application.

A key reason for this approach is the inconsistency in package usage across applications and the lack of visibility into which projects rely on a particular package. Updating a package often entails significant effort to update all dependent applications, creating a disincentive to modify shared packages. Furthermore, the current system does not support managing multiple versions or branches of a package, further complicating updates and the rollout of significant changes.

The choice to drop packages was a recurring theme: either an existing package nearly met the desired functionality but fell short, or the effort to create, maintain and update a new package was deemed too cumbersome. Without an established process for managing, updating and standardizing package usage, developers often opted for short-term efficiency by manually replicating and modifying code to fit immediate needs.

That said, many packages are effectively utilized and maintained. In Angular, reusable components such as input fields, order-status fields, spinners, progress bars, modals, payment modules and notification elements are commonly adopted. For example, EDS has packaged elements of the order status page, enabling consistent updates and styling across applications.

Table 2
16 most duplicated C# .NET files/components (anonymized). The 'Projects' column indicates the number of projects with files containing duplicated lines, as detected by SonarQube.

File ID	Projects	Notes
File N	61	Contains application configuration settings. Minor changes and bug fixes across projects prevent detection in all cases.
File O	61	Handles request redirection to Angular. Slight differences across projects result in partial detection.
File P	59	Manages requests to the `StatusAPI` to check application health. Variations in implementation prevent detection in all projects.
File Q	55	Includes utility functions. Changes and differences between implementations account for partial detection.
File R	54	Wrapper class that stores results, warnings and errors. Updated versions are too small for duplication detection in some projects.
File S	53	Serves as the dependency registration file. Present in all projects, though variations reduce duplication detection accuracy.
File T	50	Older versions of this file are too small to trigger duplication detection but are present across multiple applications.
File U	42	Part of the controller code often overlaps, but other sections vary significantly, reflecting service-specific order handling.
File V	42	Exists in all projects but appears in different forms—sometimes as an interface or duplicated within project processes.
File W	32	Handles global application events. Variations in implementation appear across all projects.
File X	32	Used in projects implementing the old StructureMap method for dependency injection. Other projects use the `DefaultDependencyResolver`, which is also duplicated.
File Y	30	Supports StructureMap dependency injection.
File Z	30	Initializes and configures the new dependency injection mechanism.
File AA	30	Represents the new dependency injection resolver. Differences in versions and types exist across projects.
File AB	25	Submits orders to the backend. Varies across projects to accommodate service-specific requirements, such as handling payment, files, or additional checks.
File AC	21	Provides methods for order registration and uniqueness checks. While uniqueness criteria vary by service, the core functionality remains similar across implementations.

A similar pattern is observed in C# .NET development. Lower-level classes, such as those for authentication, payment, logging, relay and service bus connections and HTTP clients for zip codes and countries, are frequently packaged. However, higher-level process classes, such as those handling zip code or country retrievals that lack application-specific logic, remain unpackaged. Likewise, processes and controllers with minimal application-specific logic have not been abstracted into reusable packages, despite the feasibility of doing so through base classes or interfaces.

5.2. Impact and risk analysis

To qualitatively assess our changes, we determined the associated 'impact' and 'risk' of packaging the duplicated code. We define the impact as the effect on the technical debt, e.g., how much the code was duplicated across projects. Files that have been duplicated between two projects are unlikely to have a great impact, as there is not that much duplication; only once. Code that is duplicated over six times (about ten percent of the projects currently) are considered as medium impact. This is the internal norm that the developers of EDS currently use, at which point the component should be refactored into a central repository. It is also a clear sign that the functionality is needed by multiple projects, but the code is not properly reused. Lastly, we also consider files containing duplicated code with a high impact, over 31 projects (half the projects currently). This indicates a lot of duplication that is hard to maintain and evolve simultaneously and consistently. We define the 'risk' as the magnitude of application-specific code, which we can categorize into four different levels: the code is already packaged, it contains no application-specific functionality, it contains optional functionality configurable by parameters or other dependencies, or the component is predominantly specific to the application at hand. This is summarized in Table 3.

Table 3
Definition of impact and risk.

Impact	In # (%) of applications
Low	0–6 (0%–10%)
Medium	7–31 (10%–50%)
High	32–62 (50%–100%)
Risk	**How application-specific**
Very low	Seen before/already packaged
Low	Copy–paste without application-specific code
Medium	Copy–paste with application-specific parameters/dependencies
High	Application-specific code

5.3. Selection of candidates to refactor

The selection of the two applications to refactor (EDS.EBA and EDS.PAS) was guided by the impact and risk analysis outlined above, and to ensure maximum benefits from the intervention while minimizing complexity. Impact was assessed based on the extent of cross-project duplication, where components duplicated in a significant number of applications were identified as critical targets.

EDS.EBA, the system for accredited company registration, was chosen due to its high levels of duplication in components shared across applications, such as `HealthProbeController` and `ProcessResult`, which were low-risk candidates for refactoring due to their

Table 4
Top Angular applications by duplication impact.

#	Subsystem	Total appearances	Number of subsystems shared with	Weight
1	EDS.EBA	27	47	1,269
2	EDS.AVD	23	47	1,081
3	EDS.SBD	23	47	1,081
4	EDS.BPA	20	47	940
5	EDS.ERK	20	46	920
...				
35	EDS.PAS	9	42	378

Table 5
Top C# applications by duplication impact.

#	Subsystem	Total appearances	Number of subsystems shared with	Weight
1	EDS.EBA	20	54	1,080
2	EDS.PAS	18	52	936
3	EDS.AVD	17	55	935
4	EDS.SBD	17	54	918
5	EDS.DOM	16	54	864
...				

Table 6
Duplicated .NET components in EDS.EBA before refactoring (Anonymized).

Component ID	Duplicated lines
File AD	83
File V	9
File S	41
File AE	44
File AB	27
File O	50
File X	21

Table 7
Duplicated Angular components in EDS.EBA before refactoring (Anonymized).

Component ID	Duplicated lines
File C	13
File D	14
File F	40
File B	18
File A	16

reusable nature. EDS.PAS, used for vehicle inspection scheduling, complemented this by including components with medium duplication levels and manageable application specificity, enabling the intervention to demonstrate scalability across varying conditions of impact and risk.

This dual focus on impact and risk ensured that the selected applications not only served as effective proofs of concept but also addressed key areas of technical debt, delivering organizational benefits that extended beyond the scope of this study.

Quantitative basis for candidate selection

To provide a quantitative foundation for selecting EDS.EBA and EDS.PAS, we analyzed duplication metrics across all 62 applications in the EDS system. For each application, we calculated its *Total Appearances* (the number of files where it appears as a duplicated instance) and the *Number of Subsystems Shared With* (the number of unique applications sharing at least one file with it). A *Weight* metric, computed as the product of these two values, was used to quantify the *impact* strength of duplication. Tables 4 and 5 present the top five applications from the Angular and C# datasets, respectively, highlighting the positions of EDS.EBA and EDS.PAS.

In the Angular dataset (Table 4), EDS.EBA exhibits the highest impact with a Weight of 1269, appearing in 27 files and sharing with 47 subsystems, confirming its classification as a high-impact candidate. EDS.PAS, with a weight of 378 (9 appearances, 42 shared subsystems), ranks mid-tier among the 62 applications, aligning with a medium-impact designation. In the C# dataset (Table 5), EDS.EBA again leads with a Weight of 1080 (20 appearances, 54 shared subsystems), while EDS.PAS follows closely with a Weight of 936 (18 appearances, 52 shared subsystems), positioning it near the top but still reflective of medium-to-high impact relative to the full set. The metrics point to the selection of EDS.EBA for its extensive duplication (high impact) and EDS.PAS for its significant yet more moderate duplication (medium impact), with a balanced evaluation of refactoring feasibility and benefits.

6. Phase III: Action

In this phase, we implemented a systematic refactoring to extract shared components, reduce duplication and enhance reusability across the two RDW applications.

6.1. Refactoring EDS.EBA and EDS.PAS

EDS.EBA had significant code duplication across its backend (.NET) and frontend (Angular) components. Table 6 shows the key duplicated .NET components, while Table 7 highlights the Angular components.

EDS.PAS exhibited significant duplication across its .NET and Angular components, similar to EDS.EBA. Table 8 presents the .NET components flagged for duplication, while Table 9 highlights the Angular components.

The refactoring process primarily targeted Type I and Type II clones (Rattan et al., 2013), which were directly replaceable with reusable components. For Type III clones, a more detailed analysis was required to generalize the functionality while preserving application-specific logic. This involved breaking down modified clones into modular functions with configurable parameters, ensuring flexibility while maintaining code reuse. An example of this approach can be seen in the refactoring of the `OrderStatus` component, where application-specific variations needed parameterized adjustments, rather than a simple extraction.

Despite the significant reduction in code duplication, refactoring challenges persisted, particularly in the Angular code, where further architectural modifications were necessary to handle minor customizations. Additionally, the refactoring process was delayed by out-of-date dependencies, which needed to be resolved before refactoring could proceed effectively.

6.2. Observations on clone types during refactoring

During the refactoring of EDS.EBA and EDS.PAS, we categorized duplicated code according to the clone taxonomy (Rattan et al., 2013) to assess how different types influenced our approach and outcomes. Type I clones, such as exact copies of utility functions (e.g., File Q in Table 2, duplicated in 55 projects) and standardized page components (e.g., File A in Table 1, in 60 projects), were the most prevalent. These were straightforward to refactor by extracting into shared NPM and NuGet packages, yielding immediate reductions (e.g., 83 lines from File AD in EDS.EBA, Table 7).

Type II clones, such as renamed dependency injection configurations (e.g., File S in 53 projects), required minor normalization before packaging, contributing to a 41% duplication reduction in EDS.EBA. Type III clones, like the OrderStatus component (File F, duplicated in 29 projects), exhibited near-miss duplication with application-specific modifications (e.g., custom fields). Refactoring these involved modular functions with configurable parameters, increasing initial effort (e.g., 40 lines in EDS.EBA, Table 7) but enabling reuse. This occasionally increased integration complexity, though offset by maintainability gains. Type IV clones were rare and not addressed, requiring redesign beyond our scope.

Table 8
Duplicated .NET components in EDS.PAS before refactoring (Anonymized).

Component ID	Duplicated lines
File AF	50
File AE	27
File AG	35
File P	26
File AH	14
File AI	84
File AA	65
File X	27

Table 9
Duplicated Angular components in EDS.PAS before refactoring (Anonymized).

Component ID	Duplicated lines
File C	16
File E	25
File AJ	13
File AK	13
File F	22
File A	16

7. Phase IV: Observation

This section presents the results of the two intervention cycles conducted on the EDS.EBA and EDS.PAS applications within RDW's Electronic Services System (EDS). The results are reported in terms of code duplication reduction, code reuse improvements and developer feedback. We also highlight the efficiency gains observed between the two cycles.

7.1. Cycle 1: refactoring EDS.EBA

The first cycle took overall 109 h, and focused on the refactoring of the EDS.EBA (Erkend Bedrijf Aanmelden) application, which handles the accreditation of companies. The application suffered from significant cross-project code duplication, especially in components like order processing, contact forms and validation features. Using SonarQube, we identified key areas where code was duplicated across multiple applications.

Code duplication reduction: The pre-refactoring analysis revealed 2605 lines of duplicated code in the EDS.EBA application. Following the refactoring effort, this was reduced to 1455 lines, resulting in the elimination of 1150 duplicated lines, representing a 44% reduction (see Table 10). Most of the refactored code was packaged into reusable NPM (for Angular code) and NuGet (for .NET code) libraries, focusing on common features such as user authentication, validation and retry policies. Table 11 summarizes the main components refactored during this cycle. The refactoring efforts removed approximately 1000 duplicated lines of code from the EDS.EBA application. Although time-intensive due to the need to establish reusable structures, this cycle provided a foundational process for subsequent interventions.

Code reuse: Post-refactoring, we observed a 33% improvement in code reuse across multiple projects. Shared components such as 'Header', 'OrderStatus' and 'AkkoordPage' were extracted into reusable packages, which were then integrated into other applications. This shift allowed for easier maintenance and reduced the risk of future code duplication.

The reported refactoring time (109 h) included not only the direct removal of duplicated lines, but also several other essential activities, such as:

- Identifying and analyzing duplicated code across applications,
- Designing reusable components and ensuring their compatibility,
- Implementing refactoring while preserving functionality, and
- Testing and validating the refactored components.

Table 10
(Duplicated) lines before and after refactoring EBA.

	Before	After
Lines of code	9,373	8,936
Duplicated lines (#)	2,605	1,455
Duplicated lines (%)	27,8%	16,3%

Table 11
Refactored components in Cycle 1 (EDS.EBA). The numbers in brackets show the number of other applications where the Component was also detected.

Component	Impact	Refactoring action
File Y	High (55)	Moved to a shared package to standardize configuration handling.
File O	Medium (21)	Extracted as a reusable package to manage redirects consistently.
File S	High (50)	Centralized into a package for wrapping process outcomes.
File V	Medium (17)	Packaged with configurable defaults for retry policies.
File Y	Medium (23)	Consolidated dependency injection logic into a shared package.

Table 12
(Duplicated) lines before and after refactoring PAS.

	Before	After
Lines of code	14,139	13,591
Duplicated lines (#)	2,516	1,708
Duplicated lines (%)	17,8%	12,6%

Table 13
Refactored components in Cycle 2 (EDS.PAS). The numbers in brackets show the number of other applications where the Component was also detected.

Component	Impact	Refactoring action
File P	Medium (9)	Re-engineered to dynamically configure dependencies via constructor parameters.
File U	Medium (6)	Partial refactoring to extract reusable logic for order submission processes.
Shared elements	Low (4)	Centralized shared elements into common packages to reduce duplication.
Startup Configuration	Medium (23)	Simplified and moved shared logic into reusable helper functions.

7.2. Cycle 2: refactoring EDS.PAS

The second cycle targeted the EDS.PAS (Product Aanmelden Systeem) application, which manages the product registration process. Leveraging the lessons learned from the first cycle, the refactoring of EDS.PAS was approached with a more streamlined process. Many of the reusable components extracted during the first cycle were directly applied to EDS.PAS, reducing both the time and complexity of the intervention. Table 13 outlines the key components addressed in this cycle (see Table 12).

Code duplication reduction: Before refactoring, EDS.PAS had 2516 lines of duplicated code. After refactoring, this was reduced to 1708 lines, eliminating 808 lines of duplication (an overall 32% reduction). Given that many components had already been refactored in the first cycle, the refactoring process for EDS.PAS was more efficient and required less manual intervention.

7.3. Comparison of the two cycles

Table 14 summarizes the results of the two refactoring cycles, demonstrating both the reduction in duplicated code and the efficiency

Table 14
Comparison between EBA and PAS on the time needed and the achieved effect.

	EBA	PAS
Components refactored	12	9
Dup. lines removed (#)	1,150	808
Dup. lines removed (%)	44.15%	32.11%
Time refactoring (h)	109	12
Avg. time/component (h)	9.1	1.3

gains realized from one cycle to the next. The data indicate that the second intervention cycle was more efficient, both in terms of time required and the ease of applying reusable components. The overall reduction in duplicated lines of code was consistent across both cycles, showing that the approach is scalable and effective for managing code duplication in large, multi-application systems.

Code reuse. The shared components from the first cycle were heavily reused in EDS.PAS. Features like form validation, order processing and error handling were integrated directly from the packages created during the refactoring of EDS.EBA. This allowed for consistent functionality across the two applications and minimized redundancy. Code reuse in EDS.PAS improved by approximately 35%.

Efficiency gains. One of the most significant outcomes of the second cycle was the time savings achieved. The experience gained during the refactoring of EDS.EBA directly influenced the approach to EDS.PAS. Refactoring EDS.PAS was much faster (see Table 14), as many reusable components were already created and developers were more familiar with the process. This efficiency demonstrates the scalability of the approach when refactoring additional applications within the system. This cycle was completed in a shorter timeframe compared to Cycle 1: the efficiency gains were attributed to the reuse of previously created shared packages and an established workflow.

While our findings demonstrate a substantial reduction in code duplication and improved refactoring efficiency (89% faster in the second subsystem), we acknowledge that these results have not been tested for statistical significance. The nature of this industrial case study, with only two subsystems being refactored, does not provide a sufficiently large sample for rigorous statistical hypothesis testing. Instead, we rely on practical, observed improvements in maintainability, code reuse, and developer-reported efficiency gains to assess the impact of our approach. Future studies with a larger dataset could apply statistical tests to further validate these findings.

The efficiency gains between cycles were partly attributable to clone types addressed: in Cycle 1 (EDS.EBA), focusing on Type I and II clones enabled a quick extraction, reducing duplication by 44% (Table 14). The higher initial time (109 h) reflected establishing packages and handling Type III clones (e.g., OrderStatus), requiring parameterization. In Cycle 2 (EDS.PAS), reusing these components (primarily Type I and II) resulted in faster refactoring (12 h) and a 32% reduction (Table 14). Developer feedback noted Type III clones occasionally complicated integration, mitigated by Cycle 1 lessons. The rarity of Type IV clones likely enhanced result consistency, as their presence could skew maintainability metrics.

8. Phase V: Reflection

Developer feedback on Cycle 1: Developers reported that, while the initial refactoring process was time-consuming due to the need for manually extracting shared components, it significantly improved the clarity and maintainability of the code. During the cycle, the EDS team recognized that their current development approach had led to significant technical debt, exceeding initial expectations. The results of the first cycle highlighted clear opportunities for improvement and provided actionable methods to reduce existing debt.

Developer feedback on Cycle 2: Developers working on the second cycle reported an improved workflow due to the availability of reusable components from the first intervention. Familiarity with the refactoring tools and processes, such as SonarQube and packaging tools like NPM and NuGet, further reduced the complexity of the task. Feedback suggested that future applications would likely see even greater efficiency gains as more reusable components are developed and shared across the entire EDS system.

Developers noted that Type I and II clones were the easiest to refactor and integrate, with an appreciation of the immediate clarity they brought to the codebase. However, Type III clones generated mixed feedback: while their refactoring improved maintainability, some developers felt the parameterization added minor overhead to initial setup. This was also visible in the responses to the post-mortem analysis (see question 8 in the Appendix). This reinforces the need for early clone type identification to optimize refactoring workflows.

Overall reflections: The refactoring interventions provided valuable insights into maintainability and efficiency but revealed the limitations of relying solely on code-level improvements. Although the systematic refactoring successfully reduced technical debt and duplication, the management team observed that long-term benefits required a shift towards more scalable and modular system architecture.

Key lessons included the importance of early detection of duplication through tools like SonarQube, which could mitigate technical debt accumulation, and the value of engaging developers in the process to highlight the tangible benefits of refactoring. While reusable components improved collaboration and efficiency, maintaining dependencies across numerous applications posed challenges, particularly with manual updates and version control. Automating dependency management and integrating practices like microservices architecture were identified as critical next steps for achieving scalability and maintainability at an organizational level.

8.1. Post-mortem analysis: Management reflection

Six months after the refactoring was completed, a post-mortem interview was conducted with an RDW engineer and an RDW architect to assess the long-term impact of the intervention. This follow-up discussion provided more definitive takeaways regarding the feasibility and effectiveness of large-scale refactoring in RDW's system.[6] As the number of applications has now grown to 80, the key findings highlight the limitations of RDW's previous development approach and the need for a more scalable architectural strategy.

Historically, RDW's software development relied on a *clone-and-own* approach, where existing applications were duplicated and modified to meet new requirements. While this method initially enabled rapid development, the post-mortem analysis confirmed that it does not scale well beyond approximately 20 applications. With 80 applications now in production, minor variations introduced over time made it increasingly difficult to consolidate and maintain shared functionalities efficiently. The team recognized that continuing with this approach would only exacerbate technical debt and increase maintenance overhead.

Although the refactoring process successfully identified inefficiencies and reduced duplication in selected applications, it did not lead to an immediate shift in RDW's daily workflow. The team realized that addressing these inefficiencies required more than isolated refactoring efforts; instead, a fundamental re-architecture was necessary to ensure long-term scalability and maintainability. Additionally, while refactoring reduced redundancy, applications functioned the same way post-refactoring, providing little direct business value. Given the scale of RDW's software landscape, refactoring all applications would be prohibitively time-consuming, with diminishing returns. This led the

[6] This and the end-of-cycle discussions were based on the questions provided in the Appendix.

team to conclude that transitioning to a more modular, service-oriented architecture would be a more effective long-term solution.

Maintaining dependencies across 80 applications also poses a significant challenge: the team has identified persistent issues with version control, compatibility, and manual updates, making system-wide improvements difficult. To mitigate these issues, they explored automation tools such as Dependabot[7] to streamline dependency management. However, this effort underscored the broader need for a structured and automated approach to handling shared dependencies in future system iterations.

To improve scalability, RDW engineers acknowledged the advantages of migrating to a cloud-based infrastructure and restructuring applications into microservices. The monolithic nature of the existing system had made maintenance increasingly difficult, particularly in handling frequent framework upgrades, such as Angular updates. A modular approach would allow selective updates, reducing maintenance overhead and improving overall system reliability. Rather than continuing the refactoring effort across all applications, RDW decided to focus on architectural transformation. Certain components were identified as candidates for immediate deployment as microservices, while others required more extensive restructuring to ensure compatibility with the new architecture.

The most significant outcome of the post-mortem analysis was the realization that large-scale refactoring, when attempted too late, becomes impractical and offers limited long-term benefits. Instead, the discussion emphasized the need for proactive architectural planning, with a stronger focus on early identification of code duplication and the adoption of scalable design patterns. Moving forward, RDW plans to prioritize early-stage modularization, reduce reliance on the clone-and-own approach, and implement systematic dependency management strategies. By integrating microservices and shared libraries at the architectural level, the organization expects to achieve improved maintainability, streamlined deployment processes, and long-term scalability.

9. Discussion

This section synthesizes the findings from the refactoring intervention at RDW and extends them into broader implications, lessons, and challenges for software practitioners. The discussion is directly informed by the answers to the research questions (RQs): RQ1 revealed a 34% average reduction in duplicated code (41% in the EDS.EBA application and 29% in the EDS.PAS application), highlighting the refactoring's impact on maintainability and technical debt; RQ2 quantified efficiency gains (e.g., 89% faster refactoring in Cycle 2) and code reuse improvements (33%–35%), demonstrating measurable benefits; and RQ3 showed how experience from one application enhanced subsequent interventions, though scalability limits emerged as the system grew. These results frame the following analysis, connecting specific outcomes to practical strategies and long-term considerations for managing code duplication in Software Product Line (SPL) environments.

9.1. Addressing the research questions

Refactoring led to several notable improvements in code quality, maintainability, and efficiency. One of the most significant outcomes was the reduction of technical debt, as duplicate code was systematically removed. This minimization of redundancy lowered maintenance overhead and reduced the risk of inconsistencies across applications.

Another key benefit was the increased reuse of shared functionality: this shift allowed teams to get an access to existing solutions rather than repeatedly implementing similar functionality, leading to a more structured and maintainable codebase. Efficiency also improved significantly over time, as the experience gained from the initial refactoring effort made subsequent interventions much faster and smoother.

More specifically, the presented results answer the research questions:

RQ1: How does refactoring cross-project code duplication impact code maintainability and technical debt?

In the context of the work performed, refactoring significantly improved maintainability and reduced technical debt, with a 38% reduction in duplicated code across two applications, as confirmed by the SonarQube analysis. Packaging shared functionality into reusable components streamlined maintenance, minimized inconsistencies, and facilitated faster debugging and feature updates. Developers reported improved system stability and reduced risk of defects.

RQ2: What are the measurable effects of reducing code duplication on maintainability, efficiency, and code reuse within RDW's EDS?

Refactoring led to measurable improvements:

- **Code reuse:** Increased by 33% in EDS.EBA and 35% in EDS.PAS due to the introduction of reusable components.
- **Efficiency:** The second refactoring cycle required only 12 h, compared to 109 h in the first cycle, amounting to an 89% efficiency gain.
- **Technical debt reduction:** Systematic refactoring decreased maintenance overhead and provided a sustainable approach to managing technical debt.

These results confirm a strong correlation between structured refactoring and improved software maintainability in multi-application environments.

Our observations on clone types further refine the measurable effects on maintainability, efficiency, and reuse. Type I and II clones, constituting the majority of duplications (e.g., 60–61 projects for Files A and N), were the primary drivers of the 33%–35% reuse improvement, as their extraction into shared libraries was both feasible and broadly applicable. Type III clones, though less common (e.g., File F in 29 projects), introduced variability that tempered efficiency gains in Cycle 1 but enhanced subsequent interventions by informing more flexible component designs. The rarity of Type IV clones aligns with their exclusion from refactoring efforts, suggesting that SPL environments like EDS may naturally favor syntactic over semantic duplication due to standardized architectures. These patterns indicate that the effectiveness of duplication reduction is based on the clone type distribution, with simpler clones producing quicker wins, and complex clones requiring strategic planning to balance effort and benefit.

RQ3: How does experience gained from refactoring one application influence subsequent interventions?

The second intervention (EDS.PAS) benefited from experience and reusable components developed in the first cycle (EDS.EBA), leading to faster implementation and greater efficiency. Developers reported improved familiarity with tools and processes, reinforcing the scalability of the approach.

While refactoring provides clear short-term benefits, its effectiveness diminishes at scale, that require a shift toward modularization and service-oriented architectures.

9.2. Scalability challenges and architectural shift

The post-mortem analysis confirmed that refactoring alone is not a scalable long-term solution. While RDW's historical clone-and-own approach initially supported rapid development, it became increasingly unsustainable as the number of applications grew. Beyond 20 applications, maintaining consistency across cloned codebases became challenging, and by the time the system expanded to 80 applications, refactoring efforts proved infeasible. The growing complexity of dependencies, coupled with minor variations in duplicated code, further

[7] https://github.com/dependabot

constrained the effectiveness of reuse, making large-scale consolidation unmanageable.

Recognizing these limitations, RDW has shifted its focus from refactoring individual applications to a more structured architectural transformation. Rather than relying on continuous duplication management, the organization is prioritizing the development of service-oriented components designed to prevent redundancy at a structural level. By modularizing functionality into well-defined services, RDW aims to create a more maintainable and scalable ecosystem that reduces the need for reactive interventions.

Another key aspect of this shift is the adoption of automated dependency management tools to ensure consistency across applications. Maintaining version control and compatibility across a growing software landscape has been a persistent challenge, and automation is expected to streamline this process, reducing the manual overhead associated with updates and integrations. In parallel, stricter governance mechanisms are being introduced to regulate how shared code is managed and reused. Establishing clear guidelines on shared component usage will help prevent redundant development efforts and ensure that future growth is supported by a sustainable software architecture.

This transition marks a fundamental change in how RDW approaches software evolution. Rather than relying on incremental refactoring to address duplication, the focus is now on designing systems that inherently minimize redundancy and facilitate maintainability at scale. By embedding modularity and automation into the development process, RDW aims to create a more resilient and adaptable system architecture capable of supporting long-term growth.

9.3. Implications for practitioners

The RDW case highlights several key lessons that practitioners across industries can adopt to address the challenges of managing duplicated code and technical debt in large-scale SPLs. These insights emphasize the importance of proactive strategies, governance, and scalable refactoring practices.

A major takeaway from this study is the value of integrating duplication management into the software development lifecycle. Our findings suggest that systematic refactoring, as demonstrated in the EDS environment, can significantly reduce technical debt and improve maintainability. However, as systems scale, manual refactoring becomes less feasible, underscoring the need for automated approaches.

The limitations of the *clone-and-own* approach to software development, as evidenced by RDW's experience, underscore the importance of adopting scalable alternatives. While cloning and customizing existing code can initially save time, this approach becomes unsustainable when the number of applications exceeds a certain threshold. At RDW, after the mark of 20 applications, the clone-and-own strategy led to unmanageable levels of duplication, making system-wide changes costly and error-prone. Beyond this threshold, the complexity of maintaining consistency and addressing technical debt escalates, emphasizing the need for shared components, modular architectures, and automated dependency management.

To prevent redundancy from escalating, practitioners should address code cloning early in the development lifecycle. Proactive identification and mitigation of duplication, supported by governance and modular design, can prevent the accumulation of technical debt. For large systems, semi-automated approaches that suggest refactorings to resolve cloning issues could enhance developer efficiency. Such tools, which require further investigation by the research community, would complement early intervention strategies and support scalable duplication management.

Another significant implication involves the long-term management of technical debt in Software Product Line (SPL) environments. The RDW experience underscores the need to monitor how duplication evolves over time and its impact on system complexity. Practitioners can benefit from establishing thresholds to guide decision-making on when refactoring should be prioritized versus when architectural restructuring becomes essential. Such thresholds provide a structured approach to balancing short-term efficiency with long-term sustainability, which is particularly relevant in domains with rapidly evolving requirements.

Governance mechanisms also play a pivotal role in maintaining consistency and reducing redundant development efforts across teams. Organizations should define clear policies and best practices for managing shared code and leverage automated monitoring tools to enforce these standards. Beyond technical governance, fostering collaboration between teams and encouraging a culture of proactive code reuse are essential for preventing silos and ensuring cohesive development practices. Industries with large, distributed development teams can particularly benefit from these approaches to align their efforts toward shared objectives.

Finally, refactoring should be framed as an integral component of a broader architectural strategy rather than a reactive solution to accumulated debt. Early-stage modularization, combined with structured refactoring efforts, can help organizations create adaptable systems capable of evolving sustainably. By shifting their focus from reactive maintenance to proactive design, practitioners in various domains can enhance the scalability, maintainability, and longevity of their software systems, echoing the successes observed in the RDW case.

The lessons from RDW's experience provide actionable insights for practitioners across diverse industries, helping them navigate the complexities of technical debt and duplication management while fostering more robust and sustainable software development practices.

10. Threats to validity

This section discusses potential threats to the validity of this study, categorized into internal, external, and construct validity, along with measures taken to mitigate these threats.

Internal validity. Internal validity concerns arise primarily from the potential impact of confounding variables on the observed improvements in maintainability and technical debt reduction. For instance, developer experience with specific tools or familiarity with refactoring techniques may have influenced the outcomes. To mitigate this threat, we selected applications within the same SPL environment and ensured that the development team received consistent training in refactoring practices and tool usage, such as SonarQube. Additionally, the cyclical action research methodology allowed us to isolate and observe the effects of refactoring in two sequential intervention cycles, providing a comparative baseline for evaluating improvements due to systematic refactoring rather than developer expertise alone.

External validity. A key threat to external validity involves the generalizability of the findings to other industrial SPL environments, given the study's focus on the RDW's Electronic Services System (EDS). Variability in application architectures, organizational workflows, or tooling across other domains could limit the applicability of the results. To address this, we have documented the intervention process and tools used in detail, enabling replication or adaptation to other SPL environments. However, further studies across diverse SPL contexts are recommended to validate the findings more broadly.

A key limitation of this study is the sample size, as only two applications out of 62 were selected for refactoring. While this may seem small, the selection was based on a structured impact and risk analysis to maximize relevance and feasibility. Furthermore, the refactoring process was far from trivial, as it required a total of 120 engineer-hours, with the first application alone consuming 109 h due to the complexity of restructuring duplicated code into reusable components. Given the significant effort involved, expanding the study to additional applications would have been impractical within the available resources. Instead, our study served as a feasibility assessment for large-scale refactoring within RDW. Future research could extend this work by exploring automated refactoring techniques or assessing refactoring outcomes across a broader sample.

Construct validity. Construct validity pertains to the accuracy of the constructs measured in relation to the study's objectives. In this research, maintainability and technical debt reduction were assessed through code duplication metrics and developer feedback. While these measures provide insights into code quality improvements, they may not fully capture other dimensions of maintainability, such as long-term code evolution or integration complexity. To counter this, we used multiple measures — quantitative metrics from SonarQube and qualitative feedback from developers — to create a comprehensive understanding of the refactoring's impact. Future work may expand on this by incorporating additional software quality metrics, such as cyclomatic complexity and module cohesion, to deepen the assessment of refactoring impacts.

11. Conclusion

This study demonstrates the benefits of systematically refactoring cross-project code duplication in a Software Product Line (SPL) environment. By applying an action research methodology within RDW's Electronic Services System (EDS), the intervention successfully extracted duplicated code from two applications and repackaged it into reusable components. The results showed significant reductions in duplicated code and measurable efficiency gains in the refactoring process. Developer feedback highlighted improvements in maintainability, consistency, and collaboration across projects.

However, a key lesson from this work is that refactoring efforts must be undertaken early to be effective. While the approach scaled across two cycles, RDW's post-mortem highlighted that resource constraints limit its feasibility across all applications, prompting a shift to architectural re-engineering. As RDW's system expanded to 80 applications, engineers recognized that the time and effort required to refactor the entire system outweighed the expected benefits. The opportunity for systematic refactoring had passed, making a large-scale restructuring no longer a viable option.

Moving forward, organizations facing similar challenges should focus on early identification and mitigation of code duplication. Architectural strategies such as modularization and microservices can help prevent excessive duplication from accumulating over time. Additionally, automated tools for dependency management and continuous integration can support long-term maintainability by ensuring that shared components remain up to date across multiple applications.

Future research should explore approaches that integrate automated refactoring into software development pipelines, reducing reliance on large-scale manual interventions. Investigating how technical debt evolves in SPL environments and how architectural decisions influence long-term maintainability will also be valuable. Ultimately, this study underscores the importance of proactive code quality management to prevent duplication from reaching unmanageable levels.

CRediT authorship contribution statement

Jesper van der Zwaag: Writing – original draft, Software, Investigation. **Frank Driesens:** Supervision, Project administration. **Bouwe Postma:** Supervision, Project administration. **Andrea Capiluppi:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT in order to improve the readability and the grammar of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Feedback questions

Practitioners' Perspective

1. How did refactoring cross-project code duplication impact your daily workflow?
2. What were the biggest challenges you faced during the refactoring process?
3. What key lessons or best practices would you recommend to other organizations attempting a similar refactoring effort?
4. Did refactoring the first application change your perspective on code duplication? If so, how?
5. Were there any unexpected benefits or drawbacks that were not initially anticipated?

Developer Feedback Discussion

6. What was the general sentiment among the developers regarding the refactoring process?
7. Were there any concerns about the maintainability or usability of the refactored components?
8. Did the new shared components introduce any unforeseen dependencies or integration issues?
9. How did the availability of reusable components influence your development speed in later projects?
10. Would you prefer a different approach for future refactoring efforts? If so, what would you change?

Data availability

The data that has been used is confidential.

References

Ain, Q.U., Butt, W.H., Anwar, M.W., Azam, F., Maqbool, B., 2019. A systematic review on code clone detection. IEEE Access 7, 86121–86144.
Al Dallal, J., Abdin, A., 2017. Empirical evaluation of the impact of object-oriented code refactoring on quality attributes: A systematic literature review. IEEE Trans. Softw. Eng. 44 (1), 44–69.
Almogahed, A., Omar, M., Zakaria, N.H., 2021. Empirical studies on software refactoring techniques in the industrial setting. Turk. J. Comput. Math. Educ. Vol 12 (3), 1705–1716.
AlOmar, E.A., AlOmar, S.A., Mkaouer, M.W., 2023a. On the use of static analysis to engage students with software quality improvement: An experience with PMD. In: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training, Vol. 11. ICSE-SEET, IEEE, pp. 179–191.
AlOmar, E.A., Ivanov, A., Kurbatova, Z., Golubev, Y., Mkaouer, M.W., Ouni, A., Bryksin, T., Nguyen, L., Kini, A., Thakur, A., 2023b. Just-in-time code duplicates extraction. Inf. Softw. Technol. 158, 107169.
AlOmar, E.A., Mkaouer, M.W., Ouni, A., Kessentini, M., 2019. On the impact of refactoring on the relationship between quality attributes and design metrics. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, Vol. 5. ESEM, IEEE, pp. 1–11.
Alshayeb, M., 2009. Empirical investigation of refactoring effect on software quality. Inf. Softw. Technol. 51 (9), 1319–1326.
Alves, N.S., Mendes, T.S., De Mendonça, M.G., Spínola, R.O., Shull, F., Seaman, C., 2016. Identification and management of technical debt: A systematic mapping study. Inf. Softw. Technol. 70, 100–121.
Arif, A., Rana, Z.A., 2020. Refactoring of code to remove technical debt and reduce maintenance effort. In: 2020 14th International Conference on Open Source Systems and Technologies. ICOSST, IEEE, pp. 1–7.
Basili, V.R., McGarry, F.E., Pajerski, R., Zelkowitz, M.V., 2002. Lessons learned from 25 years of process improvement. In: Proceedings of the 24th International Conference on Software Engineering. ICSE '02, ACM Press, p. 69.
Bauer, V., Hauptmann, B., 2013. Assessing cross-project clones for reuse optimization. In: 2013 7th International Workshop on Software Clones. IWSC, IEEE, pp. 60–61.

Bavota, G., Russo, B., 2016. A large-scale empirical study on self-admitted technical debt. In: Proceedings of the 13th International Conference on Mining Software Repositories. ACM, pp. 315–326.

Baxter, I., Yahin, A., Moura, L., Sant'Anna, M., Bier, L., 1998. Clone detection using abstract syntax trees. In: Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272). IEEE, pp. 368–377.

Bellon, S., Koschke, R., Antoniol, G., Krinke, J., Merlo, E., 2007. Comparison and evaluation of clone detection tools. IEEE Trans. Softw. Eng. 33 (9), 577–591.

BenIdris, M., 2020. Investigate, identify and estimate the technical debt: A systematic mapping study. SSRN Electron. J..

Borjesson, A., 2006. Improve by improving software process improvers. Int. J. Bus. Inf. Syst. 1 (3), 310.

Chaparro, O., Bavota, G., Marcus, A., Di Penta, M., 2014. On the impact of refactoring operations on code quality metrics. In: 2014 IEEE International Conference on Software Maintenance and Evolution. IEEE, pp. 456–460.

Clear, T., 2009. Researching collaborative technologies in global virtual teams: Empirical studies from an interpretive perspective. In: 2009 Fourth IEEE International Conference on Global Software Engineering, vol. 19, IEEE, pp. 358–363.

Digkas, G., Nikolaidis, N., Ampatzoglou, A., Chatzigeorgiou, A., 2019. Reusing code from StackOverflow: The effect on technical debt. In: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications, Vol. 2. SEAA, IEEE, pp. 87–91.

Dubinsky, Y., Rubin, J., Berger, T., Duszynski, S., Becker, M., Czarnecki, K., 2013. An exploratory study of cloning in industrial software product lines. In: 2013 17th European Conference on Software Maintenance and Reengineering. IEEE, pp. 25–34.

Erro-Garcés, A., Alfaro-Tanco, J.A., 2016. Action research as a meta-methodology in the management field. Int. J. Qual. Methods 19, 1–11.

Farooq, S., O'Brien, C., 2014. An action research methodology for manufacturing technology selection: a supply chain perspective. Prod. Plan. &; Control. 26 (6), 467–488.

Gasparic, M., Janes, A., Sillitti, A., Succi, G., 2015. An analysis of a project reuse approach in an industrial setting. In: International Conference on Software Reuse. Springer, pp. 164–171.

Gosain, A., Sharma, G., 2015. Static analysis: A survey of techniques and tools. In: Intelligent Computing and Applications: Proceedings of the International Conference on ICA, 22-24 December 2014. Springer, pp. 581–591.

Hamdi, O., Ouni, A., AlOmar, E.A., O Cinneide, M., Mkaouer, M.W., 2021. An empirical study on the impact of refactoring on quality metrics in android applications. In: 2021 IEEE/ACM 8th International Conference on Mobile Software Engineering and Systems, Vol. 7. MobileSoft, IEEE, pp. 28–39.

Hasanain, W., Labiche, Y., Eldh, S., 2018. An analysis of complex industrial test code using clone analysis. In: 2018 IEEE International Conference on Software Quality, Reliability and Security, Vol. 1. QRS, IEEE, pp. 482–489.

He, X., Avgeriou, P., Liang, P., Li, Z., 2016. Technical debt in MDE. In: Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, vol. 11, ACM, pp. 162–172.

Inoue, K., Jarzabek, S., Cordy, J.R., Koshke, R., 2010. Fourth international workshop on software clones (IWSC). In: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 2. ACM, pp. 465–466.

Iversen, Mathiassen, Nielsen, 2004. Managing risk in software process improvement: An action research approach. MIS Q. 28 (3), 395.

Juergens, E., Deissenboeck, F., Hummel, B., Wagner, S., 2009. Do code clones matter? In: 2009 IEEE 31st International Conference on Software Engineering. IEEE, pp. 485–495.

Kannangara, S., Wijayanake, W., 2015. An empirical evaluation of impact of refactoring on internal and external measures of code quality. Int. J. Softw. Eng. Appl. 6 (1), 51–67.

Kapser, C., Godfrey, M., 2003. A taxonomy of clones in source code: The re-engineers most wanted list. In: 2nd International Workshop on Detection of Software Clones, Vol. 13. IWDSC-03.

Kapser, C., Godfrey, M., 2005. Improved tool support for the investigation of duplication in software. In: 21st IEEE International Conference on Software Maintenance. ICSM'05, IEEE, pp. 305–314.

Kästner, C., Dreiling, A., Ostermann, K., 2013. Variability mining: Consistent semi-automatic detection of product-line features. IEEE Trans. Softw. Eng. 40 (1), 67–82.

Komondoor, R., Horwitz, S., 2001. Using slicing to identify duplication in source code. In: International Static Analysis Symposium. Springer, pp. 40–56.

Lerina, A., Nardi, L., 2019. Investigating on the impact of software clones on technical debt. In: 2019 IEEE/ACM International Conference on Technical Debt, Vol. 2. TechDebt, IEEE, pp. 108–112.

Li, Z., Lu, S., Myagmar, S., Zhou, Y., 2006. CP-Miner: finding copy-paste and related bugs in large-scale software code. IEEE Trans. Softw. Eng. 32 (3), 176–192.

Malhotra, R., Chug, A., 2016. An empirical study to assess the effects of refactoring on software maintainability. In: 2016 International Conference on Advances in Computing, Communications and Informatics. ICACCI, IEEE, pp. 110–117.

Mazinanian, D., Tsantalis, N., Stein, R., Valenta, Z., 2016. Jdeodorant. In: Proceedings of the 38th International Conference on Software Engineering Companion, vol. 1, ACM, pp. 613–616.

Monden, A., Nakae, D., Kamiya, T., Sato, S.-i., Matsumoto, K.-i., 2002. Software quality analysis by code clones in industrial legacy software. In: Proceedings 8th IEEE Symposium on Software Metrics. IEEE, pp. 87–94.

Moser, R., Abrahamsson, P., Pedrycz, W., Sillitti, A., Succi, G., 2008. A case study on the impact of refactoring on quality and productivity in an agile team. In: Lecture Notes in Computer Science. Springer Berlin Heidelberg, pp. 252–266.

Nikolić, D., Stefanović, D., Dakić, D., Sladojević, S., Ristić, S., 2021. Analysis of the tools for static code analysis. In: 2021 20th International Symposium Infoteh-Jahorina. INFOTEH, IEEE, pp. 1–6.

Pargaonkar, S., 2023. Cultivating software excellence: The intersection of code quality and dynamic analysis in contemporary software development within the field of software quality engineering. Int. J. Sci. Res. (IJSR) 12 (9), 10–13.

Petersen, K., Gencel, C., Asghari, N., Baca, D., Betz, S., 2014. Action research as a model for industry-academia collaboration in the software engineering context. In: Proceedings of the 2014 International Workshop on Long-Term Industrial Collaboration on Software Engineering. pp. 55–62.

Pino, F.J., Piattini, M., Horta Travassos, G., 2013. Managing and developing distributed research projects in software engineering by means of action-research. Rev. Fac. de Ingeniería Univ. de Antioquia (68), 61–74.

Ragkhitwetsagul, C., Krinke, J., 2019. Siamese: scalable and incremental code clone search via multiple code representations. Empir. Softw. Eng. 24 (4), 2236–2284.

Ragkhitwetsagul, C., Krinke, J., Clark, D., 2018. A comparison of code similarity analysers. Empir. Softw. Eng. 23, 2464–2519.

Rantala, L., Mäntylä, M., Lenarduzzi, V., 2024. Keyword-labeled self-admitted technical debt and static code analysis have significant relationship but limited overlap. Softw. Qual. J. 32 (2), 391–429.

Rattan, D., Bhatia, R., Singh, M., 2013. Software clone detection: A systematic review. Inf. Softw. Technol. 55 (7), 1165–1199.

Reddy, K.N., Rao, A.A., 2009. A quantitative evaluation of software quality enhancement by refactoring using dependency oriented complexity metrics. In: 2009 Second International Conference on Emerging Trends in Engineering &; Technology. IEEE, pp. 1011–1018.

Rieger, M., Ducasse, S., 1998. Visual detection of duplicated code. In: ECOOP Workshops. pp. 75–76.

Rieger, M., Ducasse, S., Lanza, M., Insights into system-wide code duplication. In: 11th Working Conference on Reverse Engineering, vol. 24, IEEE Computer Society, pp. 100–109.

Roy, C.K., 2018. Large scale clone detection, analysis, and benchmarking: An evolutionary perspective (Keynote). In: 2018 IEEE 12th International Workshop on Software Clones. IWSC, IEEE, 1–1.

Roy, C.K., Cordy, J.R., 2007. A survey on software clone detection research. Queen's Sch. Comput. TR 541 (115), 64–68.

Sampaio, S., Marinho, M., Luna, A., Moura, H., 2015. Reflecting, adapting and learning in small software organizations: an action research approach. In: International Conferences on Software Engineering and Knowledge Engineering, vol. 2015, KSI Research Inc. and Knowledge Systems Institute Graduate School, pp. 46–50.

Sano, M., Choi, E., Yoshida, N., Yamanaka, Y., Inoue, K., 2014. Supporting clone analysis with tag cloud visualization. In: Proceedings of the International Workshop on Innovative Software Development Methodologies and Practices. ACM, pp. 94–99.

Santos, P.S.M.d., Travassos, G.H., 2009. Action research use in software engineering: An initial survey. In: 2009 3rd International Symposium on Empirical Software Engineering and Measurement, vol. 14, IEEE, pp. 414–417.

Shatnawi, R., Li, W., 2011. An empirical assessment of refactoring impact on software quality using a hierarchical quality model. Int. J. Softw. Eng. Appl. 5 (4), 127–149.

Shrivastava, S.V., Shrivastava, V., 2008. Impact of metrics based refactoring on the software quality: a case study. In: TENCON 2008 - 2008 IEEE Region 10 Conference, vol. 1, IEEE, pp. 1–6.

Staron, M., 2019. Action research in software engineering: Metrics' research perspective (invited talk). In: International Conference on Current Trends in Theory and Practice of Informatics. pp. 39–49.

Stefanović, D., Nikolić, D., Dakić, D., Spasojević, I., Ristić, S., 2020. Static code analysis tools: A systematic literature review. In: Annals of DAAAM & Proceedings, vol. 31, no. 1, pp. 565–573.

Stefanović, D., Nikolić, D., Havzi, S., Lolić, T., Dakić, D., 2021. Identification of strategies over tools for static code analysis. IOP Conf. Ser.: Mater. Sci. Eng. 1163 (1), 012012.

Stroggylos, K., Spinellis, D., 2007. Refactoring–Does it improve software quality? In: Fifth International Workshop on Software Quality (WoSQ'07: ICSE Workshops 2007), vol. 8, IEEE, 10–10.

Szőke, G., Antal, G., Nagy, C., Ferenc, R., Gyimóthy, T., 2017. Empirical study on refactoring large-scale industrial systems and its effects on maintainability. J. Syst. Softw. 129, 107–126.

Szoke, G., Nagy, C., Hegedus, P., Ferenc, R., Gyimothy, T., 2015. Do automatic refactorings improve maintainability? An industrial case study. In: 2015 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 429–438.

Tang, Y., Khatchadourian, R., Bagherzadeh, M., Singh, R., Stewart, A., Raja, A., 2021. An empirical study of refactorings and technical debt in machine learning systems. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, IEEE, pp. 238–250.

Torres, R., 2004. Source Code Mining for Code Duplication Refactorings with Formal Concept Analysis (Ph.D. thesis). Citeseer.

Tsantalis, N., Mazinanian, D., Krishnan, G.P., 2015. Assessing the refactorability of software clones. IEEE Trans. Softw. Eng. 41 (11), 1055–1090.

van der Schuur, H., Jansen, S., Brinkkemper, S., 2011. If the SOK fits, wear it: Pragmatic process improvement through software operation knowledge. In: International Conference on Product Focused Software Process Improvement. Springer, pp. 306–321.

Yoshida, N., Higo, Y., Kusumoto, S., Inoue, K., 2012. An experience report on analyzing industrial software systems using code clone detection techniques. In: 2012 19th Asia-Pacific Software Engineering Conference. IEEE, pp. 310–313.

Zarour, M., Desharnais, J.-M., Alarifi, A., Habra, N., Cassiers, G., Robaeys, A., 2012. Gained experience by making intervention to improve software process in very small organizations. In: Software Process Improvement and Capability Determination: 12th International Conference, SPICE 2012, Palma, Spain, May 29-31, 2012. Proceedings 12. Springer, pp. 51–61.

Zhang, G., Peng, X., Xing, Z., Zhao, W., 2012. Cloning practices: Why developers clone and what can be changed. In: 2012 28th IEEE International Conference on Software Maintenance, Vol. 22. ICSM, IEEE, pp. 285–294.

Zheng, J., Williams, L., Nagappan, N., Snipes, W., Hudepohl, J., Vouk, M., 2006. On the value of static analysis for fault detection in software. IEEE Trans. Softw. Eng. 32 (4), 240–253.